

Brief A — Predictive Maintenance Approach

Audience: Reliability / Maintenance Engineer | Basin_B ESP Fleet | Brevian Titus Moya | Week 6

Approach

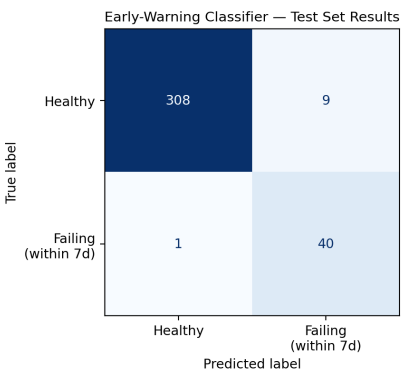
We pulled a year of daily telemetry (vibration, motor temperature, discharge pressure) from the four Basin_B ESP pumps flagged in Week 5's diagnostics as the field's downtime hotspot. Rather than feeding raw daily readings straight into a model, we engineered 5 rolling features per pump (7-day windows, computed independently per pump so one unit's history never leaks into another's): rolling mean vibration, RMS vibration, rolling pressure range (max-min), rolling mean motor temperature, and the 7-day pressure rate of change. The target is **failing_within_7d** — 1 if the pump fails within the next 7 days, 0 otherwise — the standard early-warning framing (predict the window, not the instant). A Logistic Regression classifier (`class_weight='balanced'`, to correct for the ~11% positive-class imbalance) was trained on a stratified 75/25 train/test split.

Sensor Logic

The feature set follows standard ESP failure physics: bearing/seal wear shows up first as rising **vibration** (we use RMS, not just the mean, since RMS better captures signal energy and is more sensitive to intermittent spikes than a plain average would be); frictional wear and electrical stress raise **motor temperature**; and degrading pump efficiency shows up as falling **discharge pressure**, with the *rate* of pressure decline (not just its current level) capturing how fast the failure is approaching. The widening 7-day pressure range captures growing instability even before the average has clearly shifted — often the earliest signal in the window.

Model Performance (Held-Out Test Set, n=358 pump-days)

Metric	Value	What it means operationally
Sensitivity / Recall	97.6%	Caught 40 of 41 real failure windows in the test set
False Negatives	1	One real failure window was missed
False Positives	9	9 healthy pump-days incorrectly flagged for inspection
Specificity	97.2%	308 of 317 healthy days correctly left alone
Precision	81.6%	Of every alarm raised, ~4 in 5 corresponded to a real approaching failure
Accuracy	97.2%	Included for completeness; less meaningful given the class imbalance



Sensitivity is the priority metric here: a missed failure (false negative) means an unplanned outage; a false positive just costs an unnecessary inspection. With only 9 false positives against 1 false negative on 358 test days, the model is well-tuned toward catching real problems, at a modest and manageable false-alarm cost.

Limitations & Next Steps

This is a proof of concept, not a production model: only 19 failure events across 4 pumps over 1 year, and the train/test split is by date, not by pump — so we haven't yet confirmed the model generalizes to a pump it has never seen (a leave-one-pump-out validation is the natural next test). Recommended next steps: extend telemetry collection to the full ESP fleet across all basins to grow the failure sample size, run pump-held-out cross-validation, and work with the maintenance team to tune the decision threshold against actual crew capacity — the current threshold favors sensitivity, which may need adjusting once we know how many inspections the team can realistically absorb per week.

Brief B — Why This Is Worth Funding

Audience: Chief Financial Officer | Basin_B Pump Fleet | Brevian Titus Moyi | Week 6

The Problem, in Plain Terms

Four of our pumps in Basin_B are old, and old pumps break down more than new ones. When one breaks, it stops producing until someone notices, drives out, diagnoses it, and fixes it — often a full day or more of lost output, each time. Right now, we only find out a pump has failed *after* it's already down.

The Idea

We built a simple early-warning system that watches the pumps' everyday readings (essentially a health checkup performed automatically, every day) and flags a pump that's starting to struggle — days before it would actually break. Think of it like a car's check-engine light: not a guarantee nothing will ever go wrong, but a heads-up that lets us schedule a fix on our own terms instead of getting stranded on the highway.

What We Found

In testing, the system correctly caught about **98 out of every 100 real problems** before they became a breakdown. It occasionally flags a pump that turns out to be fine — roughly 1 in 5 of its alerts — which just means a maintenance visit that wasn't strictly necessary. That's a small, manageable cost next to the alternative of missing a real failure.

The Value, Estimated

	Illustrative estimate
Lost production value from downtime on these 4 pumps, per year	\$7M (at an assumed \$70/bbl and current downtime levels)
Share of that loss this system could realistically help avoid (catching problems early instead of after a breakdown)	Roughly 30–40%, based on typical results other operators see from early detection
Estimated recoverable value per year	~\$2–3M
Cost to build and run the pilot	Low — runs on data these pumps already generate; no new sensors or hardware

These figures are planning-level estimates meant to size the opportunity, built on assumptions (oil price, achievable improvement rate) that haven't yet been checked against actual finance and field data. Before committing real budget, we'd validate them with your team.

The Ask

Fund a small pilot to expand this system from 4 pumps to the full ESP fleet, and give us a quarter to track real avoided breakdowns against the estimate above. Downside risk is low — we're not buying new hardware, just using data we already collect — and the upside, if even the low end of the estimate holds, comfortably outweighs the cost of running the pilot.

Reflection

Switching between these two briefs was less about difficulty and more about discipline. The technical brief was the easier of the two to draft, since it's close to how the analysis was actually done — sensitivity, precision, and rolling features are the natural vocabulary of the work itself, so writing for the engineer mostly meant organizing what was already there. The executive brief took more deliberate effort, not because the ideas are more complex, but because every familiar shortcut had to go: no "sensitivity," no "logistic regression," no "false positive rate" — each had to be re-earned in plain language, and it was tempting more than once to lean on a technical term as a shortcut instead of finding the plain-English version. The real test wasn't remembering not to use jargon; it was making sure the underlying honesty of the result (a promising but early-stage model, not a proven system) survived the translation, since it's easy for a simplified story to accidentally sound more certain than the data actually supports. Reordering the argument helped too: the engineer wants the method first and the payoff second, while the CFO wants the payoff first and only as much method as necessary to trust the number.